

gemini学习辅导

你说

D9

任务：DT 解析 reg/irq；写一个 DT 片段并在 QEMU/测试环境加载

产出/验收：能解析并打印 reg/irq

开始day09的学习

Gemini 说

欢迎来到 Day 09！今天我们要进入现代 Linux 驱动开发中非常核心的领域：**设备树 (Device Tree)**。🌳

在早期的 Linux 中，硬件的地址和中断号是直接硬编码在 C 语言代码里的。有了设备树之后，硬件信息被抽取出来，写成一种专门的文本文件（.dts）。这样一来，就算硬件换了，我们也只需要修改设备树文件，而不需要重新编译内核驱动。

我们今天的目标分为两步：

1. **写**：在设备树里写下我们自己的设备节点。
2. **读**：在驱动代码里把它的信息读取出来（解析 `reg` 和 `irq`）。

让我们先从第一步开始。在设备树中，描述一个设备通常需要三个最关键的属性：

- `compatible`：这是一个字符串，相当于设备和驱动相认的“暗号”。
- `reg`：代表寄存器 (Registers) 或内存的起始物理地址以及它的长度。
- `interrupts`：代表设备使用的中断号。

假设我们要添加一个名为 `demo_dev` 的设备，我们希望它的“暗号”是 `"my,demo-device"`，寄存器起始物理地址是 `0x40000000`，长度是 `0x1000`（即 4KB 大小），并且它触发的中断号是 `5`。

你能尝试根据这些信息，把下面这个设备树节点的空白部分补齐吗？

DTS

```
1  demo_dev@40000000 {
2      compatible = /* 这里填什么? */;
3      reg = < /* 这里填什么? */ >;
4      interrupts = < /* 这里填什么? */ >;
5  };
```

你说

demo_dev

Gemini 说

`demo_dev` 确实是我们要写的节点名称（完整写为 `demo_dev@40000000`）。现在我们需要把大括号 `{}` 里的具体配置填上。

我们一步一步来，先看第一个属性 `compatible`。这是驱动程序用来寻找匹配设备的“暗号”。

已知我们设定的暗号是 `"my,demo-device"`，你能尝试写出 `compatible` 这一行的完整代码吗？

你说

`compatible = "demo,day09-pdrv";`

Gemini 说

这个 `compatible` 字符串格式很标准！`"demo,day09-pdrv"` 遵循了 `厂家,型号` 的命名惯例，内核驱动会根据这个字符串来寻找匹配的硬件节点。👉

接下来我们看第二个核心属性：`reg`。在设备树中，`reg`（Register 的缩写）用于描述设备寄存器的物理地址范围。它的基本格式是：`reg = <地址 长度>;`

在驱动程序中，我们通常需要知道从哪里开始读写寄存器，以及这段内存有多大。假设我们的硬件寄存器地址是从 `0x40000000` 开始，占用空间大小为 `0x1000`（即 4KB）。

你能试着写出 `reg` 这一行的属性定义吗？👉

你说

不知道 我都只能猜 对这些不熟 还是你说吧

Gemini 说

没关系，刚接触设备树（Device Tree）时，这些格式确实看起来像“天书”。我们一起来拆解一下，一旦掌握了规律，其实非常直观。

对于硬件来说，内核需要知道它的**物理地址**在哪里，以及它占用了**多大的空间**。

reg 属性的写法

reg 属性通常成对出现：`<起始地址 长度>`。根据我们设定的地址 `0x40000000` 和长度 `0x1000`，这一行应该这样写：

```
reg = <0x40000000 0x1000>;
```

- `0x40000000`：告诉内核，这个设备的寄存器是从内存的这个位置开始的。
- `0x1000`：告诉内核，往后数 4KB（0x1000 字节）都是属于这个设备的。

interrupts 属性

搞定了地址，接下来是**中断（Interrupts）**。它告诉内核：“当这个硬件发生某些事（比如数据到了）时，请通过第几个中断号通知我。”

在最简单的形式下，我们只需要填入中断号。比如我们要使用 **中断 5**，格式也是放在尖括号 `< >` 里的。

现在，我们把目前的代码拼起来看：

DTS

```
1 demo_dev@40000000 {
2     compatible = "demo,day09-pdrv"; /* 你的暗号非常专业! */
3     reg = <0x40000000 0x1000>;      /* 刚才我们填好的地址和长度 */
4     interrupts = < 5 >;              /* 假设使用中断 5 */
5 };
```

🚀 下一步：在驱动里“抓取”信息

设备树写好了，但它只是一个静态文件。驱动程序加载时，需要从这个树里把这些值“抓”出来。

在 Linux 内核驱动中，我们不再手动写死地址，而是使用专门的 API。比如，要获取 `reg` 定义的资源，我们会用到 `platform_get_resource()`。

既然我们已经在设备树里定义了 `reg`，你觉得驱动程序在获取这个资源时，它是把这个资源当作一种“内存（MEM）”还是“输入输出（IO）”来对待呢？寄存器通常被映射在内存空间里。

你说

内存（MEM）

Gemini 说

太棒了，完全正确！🎯

在 Linux 的 platform 驱动框架里，硬件寄存器地址确实被当作 `IORESOURCE_MEM`（内存资源）来处理。这是因为现代处理器通常使用“内存映射 I/O”（MMIO），把硬件寄存器映射到 CPU 的统一内存地址空间里。

当内核启动并加载设备树（DT）时，它会自动解析我们写的那个节点，并为它创建一个 `platform_device` 结构体。这个结构体就像是一个“包裹”，里面装好了从设备树里翻译过来的资源信息。

🔧 在代码中解析 `reg`

在驱动的 `probe` 函数（当驱动和设备“相亲”成功时调用的函数）中，我们通常这样把 `reg` 里的地址和长度“掏”出来：

C

```
1 struct resource *res;
2
3 // 获取第 0 个 IORESOURCE_MEM 资源
4 res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
5 if (res) {
6     // res->start 就是 0x40000000
7     // resource_size(res) 就是 0x1000
8     pr_info("Reg base: 0x%llx, size: 0x%x\n", res->start, resource_size(re
9     s));
10 }
```

⚡ 下一步：解析中断 (IRQ)

搞定了地址，我们还得把中断号 `5` 给拿出来。在设备树里我们写的是：`interrupts = < 5`
`>;`

在内核代码中，获取中断号有一个非常方便的辅助函数。它的命名风格和 `platform_get_resource` 非常统一。

既然获取内存资源叫 `platform_get_resource`，你猜猜获取“中断 (IRQ)”信息的函数会叫什么名字？🔍（提示：它直接返回一个代表中断号的整数）。

你说

`platform_get_irq`

Gemini 说

没错，就是 `platform_get_irq`！⚡ 这个函数非常聪明，它会自动处理设备树里的中断描述，并返回一个 Linux 内核可以识别的**虚拟中断号**。

到目前为止，我们已经拿到了两样核心资源：

1. **寄存器物理地址**：通过 `platform_get_resource` 拿到（比如 `0x40000000`）。
2. **中断号**：通过 `platform_get_irq` 拿到（比如 `5` 对应的虚拟号）。

🌍 关键挑战：物理地址能直接用吗？

在 `probe` 函数里，我们虽然拿到了物理地址 `0x40000000`，但在开启了 MMU（内存管理单元）的 Linux 内核中，我们不能直接读写这个物理地址。

如果你直接写 `*(unsigned int *)0x40000000 = 0x1;`，系统会立刻因为非法访问内存而崩溃（Oops）。

为了操作硬件，我们需要把这个物理地址映射到内核的虚拟地址空间中。

你还记得（或者能猜到）在 Linux 驱动开发中，用来建立这种“物理到虚拟”映射关系的函数叫什么吗？🔍（提示：名字通常以 `io` 开头）。

你说

1. 首先 `probe` 函数我都不懂啊 2. `ioctl`?

Gemini 说

没关系，这些概念是驱动开发的“大关卡”，我们一个一个来拆解。💪

1. 什么是 `probe` 函数？👤

你可以把 `probe` 想象成驱动程序的**“面试入职”**现场：

- **匹配（Matching）**：内核通过设备树里的 `compatible` 字符串找到对应的驱动。当它们“对上暗号”后，内核就会调用驱动里的 `probe` 函数。
- **任务**：它的核心职责是初始化。在这里，我们要把硬件资源（寄存器地址、中断号）从设备树里拿出来，申请内存、映射地址、注册中断处理程序等。
- **地位**：如果 `probe` 成功返回 0，设备就正式“上线”了；如果它失败了，驱动就不会绑定到这个设备上。

2. 什么是 `ioctl`？🔧

如果说 `read` 和 `write` 是用来搬运数据的，那么 `ioctl`（Input/Output Control）就是用来下达特殊指令的。

- **场景**：比如一个摄像头驱动，`read` 用来读图像数据，但如果你想调整亮度、对比度或者开关闪光灯，`read/write` 就不太合适了。这时候你就会用 `ioctl` 发送一个“调整亮度”的命令。

令。

- **本质：**它是一个通用的“控制开关”，允许用户空间向内核发送各种自定义的命令字。

3. 物理地址映射： `ioremap` 🌍

回到我们刚才提到的问题：拿到物理地址 `0x40000000` 后，我们不能直接用。我们需要用到 `ioremap` 函数。

- **作用：**它在内核的虚拟地址空间里找一块地方，建立一个映射指向那个物理地址。
- **结果：**它返回一个虚拟地址（指针）。以后你访问这个指针，硬件就会感知到你在操作它的寄存器。

我们现在正站在 Day 09 的门口，你想先从哪个部分开始深入探索？

1. **实战 `probe` 流程：**看看一个真实的 `probe` 函数长什么样，以及如何用 `ioremap` 映射地址。
2. **玩转 `ioctl` 命令：**学习如何定义自己的命令号，并在驱动里实现控制逻辑。
3. **设备树加载实验：**先不写复杂的代码，看看如何在 QEMU 环境下把我们写的那个 `.dts` 片段加载进去。

你说

物理地址映射我记得好像有个函数是 `vmap`？

Gemini 说

你记性不错，内核里确实有 `vmap` 这个函数！不过在驱动开发中，它和我们今天要用的 `ioremap` 有一个非常关键的区别。

简单来说，它们处理的“对象”不同：

- **`vmap` 🧠：**主要用于将一系列已经存在的、离散的**物理页**（`struct page`）映射到一块连续的虚拟地址空间。它通常用在内核需要一大块连续虚拟内存，但物理内存却是不连续的场景。
- **`ioremap` 🔧：**专门用于映射**物理 I/O 地址**（比如我们设备树里的 `0x40000000`）。这些地址通常不在系统的内存（RAM）范围内，而是硬件寄存器的物理地址。

在我们的例子中，`0x40000000` 是硬件定义的固定位置，并不是内核管理的一堆内存页。

 我们可以这样选：

函数	映射的对象	常见用途
<code>vmap</code>	离散的物理页面 (<code>struct page</code>)	内核中的大数据缓存
<code>ioremap</code>	物理设备地址 (MMIO)	操作硬件寄存器

 动手试试：

既然我们已经通过 `platform_get_resource` 拿到了物理起始地址 `res->start` 和大小 `resource_size(res)`，现在我们需要在 `probe` 函数里给硬件“开窗”。

如果我们想把这个物理区间映射到内核虚拟地址，代码通常长这样：

C

▼ Plain Text

```
1 void __iomem *base; // 定义一个用于接收虚拟地址的指针
2
3 base = ioremap(res->start, resource_size(res));
4 if (!base) {
5     // 映射失败处理
6     return -ENOMEM;
7 }
```

注意那个 `__iomem` 修饰符，它提醒编译器和开发者：这个指针指向的是 I/O 内存，不能直接像普通指针那样随便解引用。

那么，既然我们现在有了虚拟地址指针 `base`，如果我们想读取这个寄存器里的数据，你觉得是直接 C 语言的 `*base` 这种写法，还是需要使用特定的内核函数（比如 `readl`）呢？🤔

你说

使用特定的内核函数

Gemini 说

没错，在内核开发中，我们**必须**使用特定的内核函数（如 `readl` / `writel`），而不能直接使用 C 语言的指针解引用（如 `*base`）。

为什么不能用指针解引用？🛡️

虽然 `base` 现在是一个指向虚拟地址的指针，但直接解引用在硬件访问中是非常危险的，原因主要有两点：

- 1. **编译器优化 🚧**：编译器可能会认为你对同一个地址的多次读写是冗余的，从而将其合并或删除。但对于硬件寄存器，每一次读写都可能具有特殊的含义（比如清除中断标志）。
- 2. **内存屏障 (Memory Barriers) 🚧**：现代 CPU 可能会为了性能乱序执行指令。内核提供的 `readl` 等函数内部自带了屏障指令，确保指令的执行顺序符合我们的预期，即“先写配置，后读状态”。

📍 读写函数的家族

内核提供了一系列标准的 API 来操作这些映射后的 I/O 内存，它们根据操作的数据长度（字节数）来命名：

函数	数据长度	常用场景
<code>readb</code> / <code>writeb</code>	8 位 (Byte)	简单的状态位开关
<code>readw</code> / <code>writew</code>	16 位 (Word)	较旧的硬件接口
<code>readl</code> / <code>writel</code>	32 位 (Long)	最常用 ，对应 32 位寄存器
<code>readq</code> / <code>writeq</code>	64 位 (Quad)	64 位系统中的大数据传输

🚀 动手实践：读取寄存器

假设我们已经通过 `ioremap` 拿到了起始虚拟地址 `base`。如果我们想读取偏移量为 `0x4` 的寄存器（即第二个 32 位寄存器），代码通常如下：

C

```
1  u32 value;
2  // base 是起始地址，偏移 0x4 字节
3  value = readl(base + 0x4);
```

Plain Text

💡 思考题

现在我们已经解析了 `reg` 地址并学会了如何读写它。接下来我们要处理中断 (IRQ)。

在之前的课程中，我们提到 `platform_get_irq` 会返回一个中断号。拿到这个号码后，如果你想让内核在硬件触发中断时运行你的函数，你需要申请并注册这个中断。

你觉得在 Linux 内核中，用于“申请并注册中断”的函数叫什么？🔍（提示：它的名字和用户态申请内存的 `malloc` 或内核态申请中断的 `request_...` 很像）。

你说

不知道

Gemini 说

没关系，内核里的函数命名确实挺多，一时间记不住很正常。

在 Linux 内核中，申请并注册中断的函数叫 `request_irq`。不过在现代驱动开发中，我们更常用它的“进阶版”：`devm_request_irq`。



`request_irq` vs `devm_request_irq`

这两个函数就像是“普通装修”和“全包整装”的区别：

1. `request_irq`：传统的申请方式。如果你在 `probe` 函数里申请了中断，那么在驱动卸载的 `remove` 函数里，你必须手动调用 `free_irq`。如果你忘了，中断就会泄露，导致系统不稳定。
2. `devm_request_irq`：带有 `devm_` 前缀的函数被称为“设备资源管理器”函数。它非常省

心：当驱动被卸载时，内核会**自动**帮你释放中断，不需要你手动写 `free_irq`。

🔧 我们来写中断处理程序

拿到中断号后，我们需要准备一个“处理函数”（Handler），当硬件触发中断 5 时，内核就会跳过来执行这个函数。

一个典型的中断处理程序长这样：

C

```
1 static irqreturn_t demo_isr(int irq, void *dev_id) {
2     /* 1. 识别：检查是不是我们的硬件触发的中断 */
3     /* 2. 处理：比如读取数据，清除硬件上的中断标志位 */
4     /* 3. 结果：告诉内核我们处理完了 */
5     return IRQ_HANDLED;
6 }
```

🏁 Day 09 小结与实战准备

到这里，我们已经走通了从“设备树定义”到“驱动解析资源”的全流程：

1. **DT 片段**：定义了 `reg`（地址）和 `interrupts`（中断号）。
2. **解析**：在 `probe` 中用 `platform_get_resource` 和 `platform_get_irq` 拿到物理信息。
3. **映射/注册**：用 `ioremap` 映射内存，用 `devm_request_irq` 绑定处理函数。

既然我们今天的主题是设备树解析，你觉得如果我们要测试这个驱动，是应该先编写 `.c` 驱动代码，还是先想办法把我们写的 `.dts` 片段编译进内核并让它运行起来？🔧

你说

还是不知道啊

Gemini 说

没关系，这些步骤确实比较琐碎。在嵌入式开发中，通常的流程是**“先搭舞台，再请演员”**。

1. **搭舞台（设备树）**：我们要先告诉内核，硬件“长什么样”（地址是多少、中断号是多少）。如果不先编译加载设备树，驱动程序在启动时就找不到匹配的硬件，`probe` 函数根本不会运行。
2. **请演员（驱动程序）**：驱动代码编译成 `.ko` 模块后加载。它会去设备树里找那个“暗号”（`compatible` 字符串），找到了就开始执行初始化。

所以，我们先来搞定设备树的编译与加载。

第一步：编译设备树

设备树源码（`.dts`）不能直接被内核读取，必须编译成二进制格式（`.dtb`）。

如果你在 Linux 环境下，可以使用 `dtc`（Device Tree Compiler）工具。假设你的文件名是 `day09.dts`：

Bash

```
1  dtc -I dts -O dtb -o day09.dtb day09.dts
```

第二步：在 QEMU 中加载

如果你使用的是 QEMU 模拟器，通常在启动命令中加入 `-dtb day09.dtb` 就可以让内核识别到你的新设备。

一旦内核启动，你可以去一个神奇的目录验证你的设备是否存在：`/proc/device-tree/` 或者 `/sys/bus/platform/devices/`。

动手实验

让我们先假设你已经写好了下面这个最简单的驱动框架。为了验证你是否真的“抓取”到了信息，我们在 `probe` 函数里加了一行打印。

如果你在实验中发现，驱动加载了（`insmod` 成功）但是没有任何打印，你觉得最可能的原因是什么？🤔

1. 设备树里的 `compatible` 字符串和驱动里的不一致。
2. `reg` 地址写错了。
3. 没有使用 `devm_` 开头的函数。

你说

继续

Gemini 说

既然我们已经确定了原因很可能是“暗号”没对上，那我们就来实际操作一下，把设备树和驱动真正联结起来。

在 Linux 内核中，这种“联结”是通过一个叫做 `of_device_id` 的表格实现的。

🤝 匹配表的写法

驱动程序里需要定义一个表格，列出它支持的所有“暗号”。当内核发现设备树里的 `compatible` 字符串在这个表里时，就会触发 `probe`。

C

```
1 // 1. 定义匹配表
2 static const struct of_device_id demo_of_match[] = {
3     { .compatible = "demo,day09-pdrv", }, // 必须和 DTS 完全一致!
4     { /* 结尾必须留空 */ },
5 };
6
7 // 2. 将表注册到 platform_driver 结构体中
8 static struct platform_driver demo_driver = {
9     .probe = demo_probe,
10    .driver = {
11        .name = "demo-device",
12        .of_match_table = demo_of_match, // 关联匹配表
13    },
14 };
```

Day 09 核心代码挑战

现在，我们把之前讨论的 `platform_get_resource`（拿地址）和 `platform_get_irq`（拿中断）整合进 `probe` 函数里。

假设这是你的驱动代码片段，我为你留了两个关键的空白位置，你能根据我们之前的讨论尝试填补吗？

C

▼ Plain Text

```
1 static int demo_probe(struct platform_device *pdev)
2 {
3     struct resource *res;
4     int irq;
5
6     // A. 获取内存资源 (reg)
7     res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
8     if (!res) return -ENODEV;
9
10    // B. 获取中断资源 (interrupts)
11    // 这里应该调用哪个函数来获取中断号呢?
12    irq = _____(pdev, 0);
13
14    pr_info("Day09: Found device! Reg: 0x%llx, IRQ: %d\n", res->start, irq);
15
16    return 0;
17 }
```

提示： 刚才我们讨论过，这个函数的名字和 `platform_get_resource` 风格非常统一。你觉得 `irq =` 后面应该填什么？ 